



CPE3243 วิศวกรรมซอฟต์แวร์ (Software Engineering)

Piyavit Laung-Aram
Major of Computer Engineering
Faculty of Engineering
Ramkhamhaeng University, Thailand

การเขียนโปรแกรมด้วยภาษา Java ส่วนที่ 2

(Java Programming Part 2)



OOPS คืออะไร?

Object-Oriented Programming System (OOPs) เป็นแนวทางการเขียนโปรแกรมที่ทำงานบนหลักการนามธรรม การห่อหุ้ม การสืบทอด และความหลากหลาย อนุญาตให้ผู้ใช้สร้าง Object ที่ต้องการ และสร้างวิธีการจัดการกับ Object เหล่านั้น แนวคิดพื้นฐานของ OOP คือการสร้าง Object นำกลับมาใช้ใหม่ตลอดทั้งโปรแกรม และจัดการอ็อบเจกต์เหล่านี้เพื่อให้ได้ผลลัพธ์

OOP หมายถึง "การเขียนโปรแกรมเชิงออบเจกต์" เป็นแนวคิดที่เป็นที่รู้จักและใช้กันอย่างแพร่หลายในภาษาโปรแกรมสมัยใหม่เช่น Java



แนวคิดของ OOPs ใน Java พร้อมตัวอย่าง

- 1) คลาส(Class)
- 2) อ็อบเจกต์(Object)
- 3) การสืบทอด(Inheritance)
- 4) โพลีมอร์ฟิซึม(Polymorphism)
- 5) แอ็บสแตรคชัน(Abstraction)
- 6) การห่อหุ้ม(Encapsulation)
- 7) ความสัมพันธ์(Association)
- 8) การรวม(Aggregation)
- 9) องค์ประกอบ(Composition)



1) คลาส(Class)

คลาสนี้เป็นหนึ่งในแนวคิดพื้นฐานของ OOP ซึ่งเป็นกลุ่มของเอนทิตีที่คล้ายคลึงกัน มัน
เป็นเพียงองค์ประกอบเชิงตรรกะและไม่ใช่เอนทิตีทางกายภาพ ให้เข้าใจหนึ่งในแนวคิด
OOPs นี้ด้วยตัวอย่าง

หากคุณมีคลาสที่เรียกว่า "รถยนต์ราคาแพง-Expensive Cars"

- อาจมี Object เช่น Mercedes, BMW, Audi , Macerati , Toyota ฯลฯ
- คุณลักษณะ (ข้อมูล-data) อาจเป็นราคาหรือความเร็วของรถยนต์เหล่านี้
- ในขณะที่วิธีการต่างๆ(methods) ที่สามารถดำเนินการได้กับรถยนต์เหล่านี้ เช่น ขับรถ
ถอยหลัง เบรก ฯลฯ



2) อ็อบเจกต์(Object)

อ็อบเจกต์ สามารถกำหนดเป็นอินสแตนซ์ของคลาส และสามารถมีได้หลายอินสแตนซ์ของคลาสในโปรแกรม

อ็อบเจกต์เป็นหนึ่งในแนวคิด Java OOPs ที่จะบรรจุทั้งข้อมูลและฟังก์ชันซึ่งทำงานบนข้อมูลไว้ด้วยกัน ตัวอย่างเช่น แก้วน้ำ จักรยาน ปากกา โต๊ะ รถยนต์ ฯลฯ



3) การสืบทอด(Inheritance)

การสืบทอดเป็นหนึ่งในแนวคิดพื้นฐานของ OOP โดยที่อ็อบเจกต์หนึ่งได้รับคุณสมบัติและพฤติกรรมของอ็อบเจกต์หลัก โดยการสร้างความสัมพันธ์แบบพ่อแม่ลูก(Parent-Child) ขึ้นมาระหว่างสองคลาส มีกลไกที่แข็งแกร่งและเป็นธรรมชาติสำหรับการจัดระเบียบและการวางโครงสร้างของซอฟต์แวร์



4) โพลีมอร์ฟิซึม(Polymorphism)

โพลีมอร์ฟิซึม(Polymorphism) หมายถึงหนึ่งในแนวคิด OOPs ใน Java ซึ่งบอกถึงความสามารถของตัวแปร อ็อบเจกต์ หรือฟังก์ชันในการทำงานได้หลาย ๆ รูปแบบ ตัวอย่างเช่น ในภาษาอังกฤษ กริยา run มีความหมายต่างกันหลาย ๆ กรณี คุณสามารถนำไปใช้กับแล็ปท็อป การเดินเท้า และธุรกิจ ในที่นี้ เราเข้าใจความหมายของการวิ่งตามคำอื่นๆ ที่ใช้ควบคู่กัน หรืออาจกล่าวได้ว่าสั่งอย่างเดียวกันแต่ได้หลายอย่าง



5) แอ็บสแตรคชัน(*Abstraction*)

แอ็บสแตรคชัน เป็นหนึ่งในแนวคิด OOP ใน Java ซึ่งเป็นการแสดงคุณลักษณะที่จำเป็นในการใช้งานโดยไม่ต้องรายละเอียดการทำงานเบื้องหลังพื้น เป็นเทคนิคการสร้างชนิดข้อมูลใหม่ให้เหมาะสมกับการทำงานในแอปพลิเคชันนั้น ๆ เพื่อให้เข้าใจถึงแนวคิด OOPs นี้ด้วย ตัวอย่าง ในขณะที่คุณขับรถ คุณไม่ต้องไปรู้อะไรมากมายเกี่ยวกับการทำงานของระบบขับเคลื่อนหรือระบบเบรกหรือระบบอื่น ๆ ของรถยนต์ว่ามันจะทำงานอย่างไร คุณเพียงแค่อำนาจสนใจการทำงานและชิ้นส่วนที่เกี่ยวข้องในการขับรถของคุณพอ เช่น พวงมาลัย , เกียร์ , คันเร่ง , ฯลฯ.



6) การห่อหุ้ม(Encapsulation)

การห่อหุ้มเป็นหนึ่งในแนวคิด Java OOP ที่ดีที่สุด ในการห่อข้อมูลและฟังก์ชันการทำงานไว้ด้วยกัน แนวคิด OOPs ข้อนี้นี้ ตัวแปรของคลาสจะถูกซ่อนจากการมองเห็นของคลาสอื่น การเข้าถึงตัวแปรสามารถเข้าถึงได้โดยผ่านเมธอดที่อยู่ภายในคลาสนั้น ตัวอย่างเช่น ในโรงเรียน นักเรียนไม่สามารถอยู่ได้โดยไม่มีชั้นเรียน



7) ความสัมพันธ์ (Association)

ความสัมพันธ์ (Association) คือความสัมพันธ์ระหว่างอ็อบเจกต์ สองชิ้น เป็นหนึ่งในแนวคิด OOP ใน Java ซึ่งกำหนดความหลากหลายระหว่างอ็อบเจกต์ ในแนวคิด OOP นี้ อ็อบเจกต์ ทั้งหมดมีวงจรชีวิตแยกจากกัน และไม่มีเจ้าของ ตัวอย่างเช่น นักเรียนหลายคนสามารถเชื่อมโยงกับครูคนหนึ่งได้ ในขณะที่นักเรียนคนหนึ่งสามารถเชื่อมโยงกับครูหลายคนได้



8) การรวม(Aggregation)

ในเทคนิคนี้อ็อบเจกต์ทั้งหมดมีวงจรชีวิตที่แยกจากกัน เช่น อ็อบเจกต์ลูกไม่สามารถเป็นของอ็อบเจกต์แม่อื่นได้

หมายความว่า อ็อบเจกต์ลูก เกิดจากอ็อบเจกต์แม่ได้เพียงตัวเดียว ไม่สามารถเกิดจากอ็อบเจกต์แม่หลายตัวได้เช่น พิจารณาจากตัวเราเป็นอ็อบเจกต์ที่เกิดจากอ็อบเจกต์คุณแม่ แม่คุณแม่ของเราจะไม่อยู่แล้ว ตัวเราก็ยังอยู่ หมายความว่าชีวิตเรากับชีวิตแม่ของเรามีชีวิตที่อิสระจากกัน



9) องค์ประกอบ(Composition)

Composition เป็นรูปแบบเฉพาะของ Aggregation เรียกอีกอย่างว่าความสัมพันธ์ "death-ความตาย" อ็อบเจกต์ลูกไม่มีวงจรชีวิต ดังนั้นเมื่ออ็อบเจกต์หลักถูกลบอ็อบเจกต์ลูกทั้งหมดจะถูกลบไปด้วยโดยอัตโนมัติ ลองมาดูตัวอย่างกัน บ้านและห้องภายในบ้าน บ้านทุกหลังสามารถมีได้หลายห้อง ห้องหนึ่งห้องไม่สามารถเป็นส่วนหนึ่งของบ้านได้หลายหลังที่แตกต่างกันได้ ดังนั้นหากคุณลบบ้านไปห้องของบ้านก็จะถูกลบไปด้วย



ข้อดีของ OOP (ระบบการเขียน โปรแกรมเชิงวัตถุ)

- แนวคิด OOPs ใน Java นำเสนอโครงสร้างโมดูลที่เข้าใจง่ายและชัดเจนสำหรับโปรแกรม
- อ็อบเจกต์ที่สร้างขึ้นสำหรับ Object-Oriented Programs สามารถใช้ซ้ำในโปรแกรมอื่นได้ ดังนั้นจึงช่วยประหยัดต้นทุนการพัฒนาอย่างมาก
- โปรแกรมขนาดใหญ่นั้นเขียนยาก แต่ถ้าทีมพัฒนาและทีมออกแบบปฏิบัติตามแนวคิด OOPS พวกเขาจะสามารถออกแบบได้ดีขึ้นโดยมีข้อบกพร่องน้อยที่สุด
- โมดูลาร์ของโปรแกรมสามารถปรับปรุงได้ง่าย เนื่องจากทุก ๆ อ็อบเจกต์เป็นอิสระจากกัน



การเปรียบเทียบ OOPS กับรูปแบบการเขียนโปรแกรมอื่นๆ

มาทำความเข้าใจกับตัวอย่างว่า Java OOPs Concepts แตกต่างจากวิธีการเขียนโปรแกรมอื่นๆ อย่างไร

ภาษาโปรแกรมสามารถจำแนกได้เป็น 3 ประเภทหลัก

- Unstructured Programming Languages : ภาษาโปรแกรมที่มีการควบคุมแบบ sequential โค้ดจะมีการเขียนซ้ำตลอดทั้งโปรแกรม
- Structured Programming Languages : มีการควบคุมแบบ non-sequential การใช้ฟังก์ชันช่วยให้สามารถใช้โค้ดซ้ำได้
- Object Oriented Programming Languages : รวมข้อมูลและการดำเนินการเข้าด้วยกัน



มาทำความเข้าใจภาษา 3 ประเภtenี้ด้วยตัวอย่างกัน

สมมติว่าคุณต้องการสร้างซอฟต์แวร์การธนาคารที่มีฟังก์ชันเช่น

- เงินฝาก
- ถอน
- แสดงยอดคงเหลือ



Unstructured Programming Languages

ภาษาโปรแกรมที่เก่าที่สุดเป็นภาษาโปรแกรมที่ไม่มีโครงสร้าง รหัสเบื้องต้นของแอปพลิเคชันธนาคารในภาษาการเขียนโปรแกรมที่ไม่มีโครงสร้างจะมีตัวแปรสองตัวของหมายเลขบัญชีหนึ่งและอีกตัวสำหรับยอดเงินในบัญชี

```
int account_number=20;
```

```
int account_balance=100;
```

สมมติว่ามีการฝากเงิน 100 บาท

```
account_balance=account_balance+100
```

ถัดไปคุณต้องแสดงยอดเงินในบัญชี

```
printf("Account Number=%d,account_number)
```

```
printf("Account Balance=%d,account_balance)
```



Unstructured Programming Languages (ต่อ)

ตอนนี้ถอนเงินจำนวน 50 ดอลลาร์แล้ว

```
account_balance=account_balance-50
```

คุณต้องแสดงยอดเงินในบัญชีอีกครั้ง

```
printf("Account Number=%d,account_number)
```

```
printf("Account Balance=%d,account_balance)
```



Unstructured Programming Languages (ต่อ)

สำหรับการฝากหรือถอนเงินใดๆ เพิ่มเติม – คุณจะต้องเขียนโค้ดซ้ำในบรรทัดเดิมซ้ำแล้วซ้ำอีก

```
int account_number = 20;  
int account_balance = 100;
```

Unstructured Programming
same code is repeated

```
account_balance = account_balance + 100
```

```
printf("Account Number = %d", account_number)
```

```
printf("Account Balance = %d", account_balance)
```

```
account_balance = account_balance - 50
```

```
printf("Account Number = %d", account_number)
```

```
printf("Account Balance = %d", account_balance)
```

```
account_balance = account_balance - 10
```

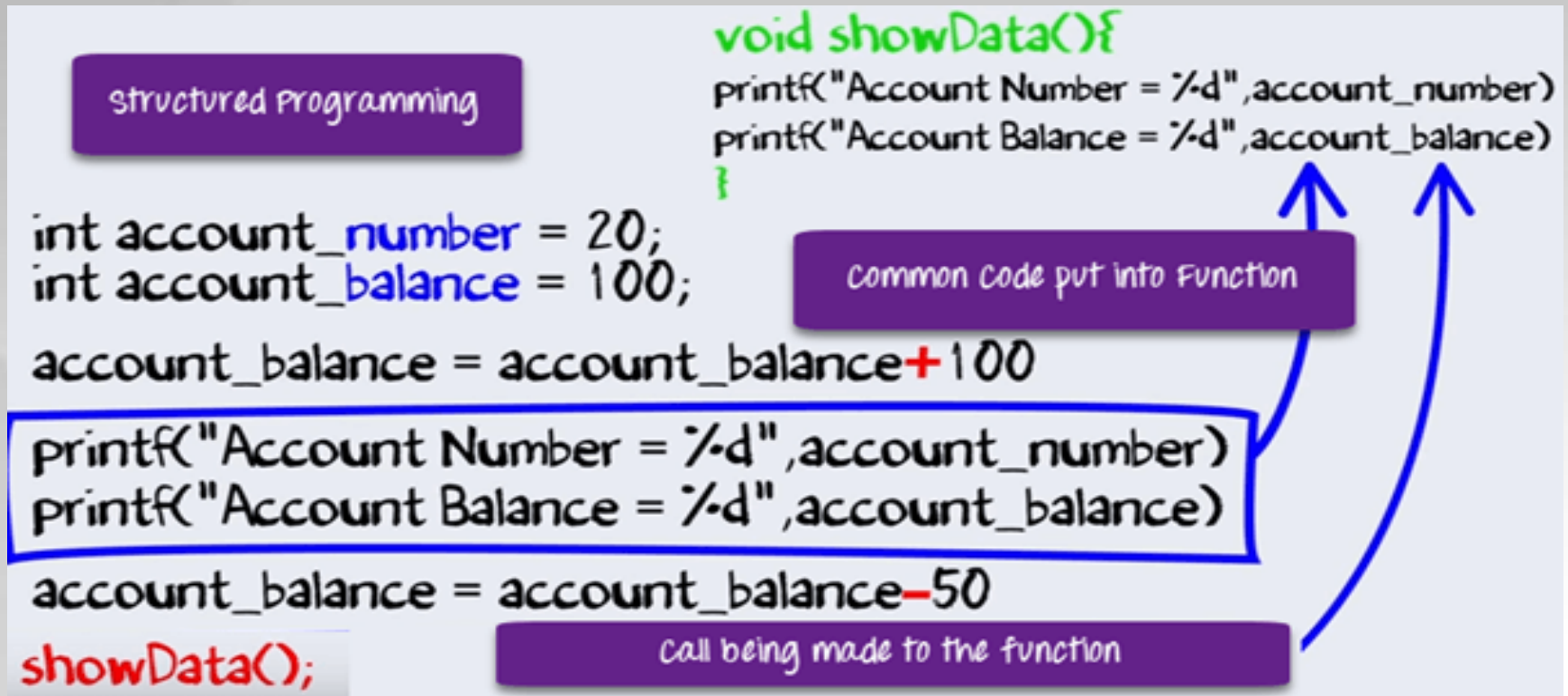
```
printf("Account Number = %d", account_number)
```

```
printf("Account Balance = %d", account_balance)
```



Structured Programming Languages

ด้วยการมาถึงของ Structured Programming บรรทัดซ้ำในโค้ดจึงถูกใส่ลงในโครงสร้าง เช่น ฟังก์ชันหรือเมธอด เรียกใช้ฟังก์ชันอย่างง่ายได้ทุกเมื่อที่ต้องการ





Object Oriented Programming Languages

ในโปรแกรมจะมีจัดการกับข้อมูลและการดำเนินการกับข้อมูล ข้อมูลและการดำเนินการกับข้อมูลนั้นเป็นคุณลักษณะพื้นฐานในโปรแกรมต่าง ๆ

ในการเขียนโปรแกรมซอฟต์แวร์ที่มีการรวมเอาข้อมูลและการดำเนินงานบนข้อมูลเข้าด้วยกัน จึงเป็นที่มาของ Object Oriented Programming ที่เรียกกันทั่วไปว่า OOPS

โค้ดใน Object Oriented Programming languages จะมีข้อมูลและการดำเนินการกับข้อมูลนั้นเป็นชุดเดียวกัน



Object Oriented Programming Languages(ต่อ)

//code 5.1

class Account

{

int account_number;

int account_balance;

public void showdata()

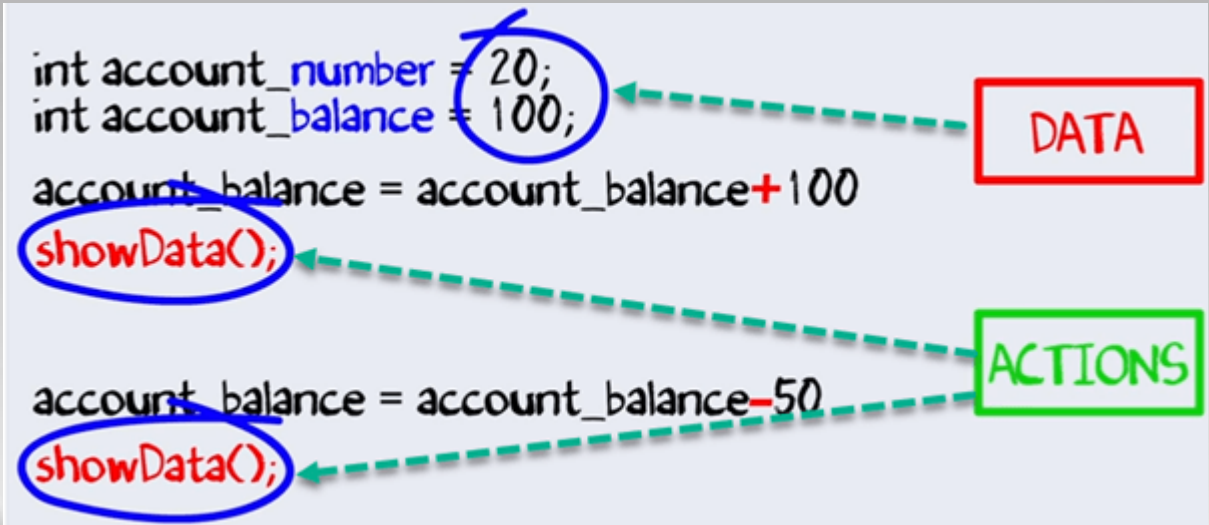
{

system.out.println("Account Number"+account_number)

system.out.println("Account Balance"+ account_balance)

}

}





Object Oriented Programming Languages(ต่อ)

การรวมข้อมูลและการดำเนินการไว้ด้วยกัน มีข้อดีมากกว่าการเขียนโปรแกรมเชิงโครงสร้าง (structural programming) แบบเดิม คือ

- Abstraction
- Encapsulation
- Inheritance
- Polymorphism

ซึ่งเราจะมาเรียนรู้ในหัวข้อ Abstraction และ Encapsulation ในเบื้องต้นก่อน



การเขียนโปรแกรมด้วยภาษา Java-2

(Java Programming)



อะไรคือ Abstraction ใน OOPs? Java Abstract Class & Method

Abstraction เป็นแนวคิดของการเขียนโปรแกรม OOP ที่ "แสดง" เฉพาะคุณลักษณะที่จำเป็นและ "ซ่อน" ข้อมูลที่ไม่จำเป็น

จุดประสงค์หลักของสิ่งที่เป็น Abstraction คือการซ่อนรายละเอียดที่ไม่จำเป็นจากผู้ใช้งาน สิ่งที่เป็นนามธรรมคือการเลือกข้อมูลจากกลุ่มที่ใหญ่กว่าเพื่อแสดงเฉพาะรายละเอียดที่เกี่ยวข้องของอ็อบเจกต์ต่อผู้ใช้งาน ช่วยลดความซับซ้อนและความพยายามในการเขียนโปรแกรม เป็นหนึ่งในแนวคิดที่สำคัญที่สุดของ OOP

มาศึกษา Abstraction ใน OOP ด้วยตัวอย่าง:

สมมติว่าคุณต้องการสร้างแอปพลิเคชันธนาคารและขอให้คุณรวบรวมข้อมูลทั้งหมดเกี่ยวกับลูกค้าของคุณ มีโอกาสที่คุณจะได้รับข้อมูลต่อไปนี้เกี่ยวกับลูกค้า



ตัวอย่าง Abstraction ใน OOPs

แต่ไม่จำเป็นต้องใช้ข้อมูลข้างต้นทั้งหมดในการสร้างแอปพลิเคชันธนาคาร

- ☒ Full Name
- ☒ Address
- ☒ Contact Number
- ☒ Tax Information
- ☒ Favorite Food
- ☒ Favorite Movie
- ☒ Favorite Actor
- ☒ Favorite Band

okay, we might
not need all these
customer
information for
a banking
application



ตัวอย่าง Abstraction ใน OOPs (ต่อ)

ดังนั้น คุณต้องเลือกเฉพาะข้อมูลที่เป็นประโยชน์สำหรับแอปพลิเคชันธนาคารของคุณ จากกลุ่มของข้อมูลนั้น ข้อมูล เช่น ชื่อ ที่อยู่ ข้อมูลภาษี ฯลฯ เหมาะสมสำหรับแอปพลิเคชันธนาคารซึ่งเป็นตัวอย่าง **Abstraction** ใน OOPs

- ☒ Full Name
- ☒ Address
- ☒ Contact Number
- ☒ Tax Information



ตัวอย่าง Abstraction ใน OOPs (ต่อ)

เนื่องจากเราได้ ดึง/ลบ/เลือก ข้อมูลลูกค้ายจากกลุ่มข้อมูลทั้งหมด กระบวนการนี้จึงเรียกว่า Abstraction ใน OOP อย่างไรก็ดีตาม ข้อมูลเดียวกันที่ดึงออกมาแล้วสามารถใช้กับแอปพลิเคชันได้หลากหลาย

ตัวอย่างเช่น คุณสามารถใช้ข้อมูลชุดเดียวกันสำหรับการสมัครงานในโรงพยาบาล แอปพลิเคชัน job portal ฐานข้อมูลของรัฐบาล ฯลฯ โดยมีการดัดแปลงเพียงเล็กน้อย หรือไม่มีเลย ดังนั้นมันจึงกลายเป็นข้อมูลหลักของคุณ นี่เป็นข้อได้เปรียบของสิ่งที่เป็น Abstraction ใน OOP



ความแตกต่างระหว่างสิ่งที่เป็น Abstraction และ Encapsulation

Abstraction	Encapsulation
Abstraction ใน Object Oriented Programming ช่วยแก้ปัญหาที่ระดับการออกแบบ(design)	Encapsulation แก้ปัญหาในระดับ ระดับการใช้ งาน(implementation)
Abstraction ใน Object Oriented Programming คือการซ่อนรายละเอียดที่ไม่ต้องการในขณะที่แสดงข้อมูลที่สำคัญที่สุด	Encapsulation หมายถึงการผูกโค้ดและข้อมูลไว้ในที่เดียวกัน
Data Abstraction ใน Java จุ่มงเน้นที่ สารสนเทศอะไรที่เอบเจกต์นั้นต้องมี	Encapsulation หมายถึง การซ่อนรายละเอียด ภายในหรือกลไกการทำงานภายในของอ็อบเจกต์ด้วยเหตุผลด้านความปลอดภัย



ความแตกต่างระหว่าง Abstract Class และ Interface

Abstract Class	Interface
abstract class สามารถมีเมธอดได้ทั้งแบบ abstract และ non-abstract	interface จะมีได้แค่ abstract method
abstract class ไม่รองรับการสืบทอดแบบ multiple inheritances	interface รองรับการใช้สืบทอดแบบ multiple inheritances
abstract class จะเตรียมการทรัพยากรต่างๆ ในการใช้งานอินเทอร์เฟซ(interface)	interface จะไม่เตรียมการทรัพยากรต่างๆ ในการใช้งาน abstract class.
abstract class มี abstract public method และวิธีการปกป้อง abstract public method	interface มีแค่เพียง public abstract methods อย่างเดียว
abstract class มีตัวแปรแบบ final, static, หรือ static final พร้อมกับการควบคุมการเข้าถึง	interface มีเพียงตัวแปร public static final



Abstract Class คืออะไร?

Abstract Class เป็นคลาสประเภทหนึ่งใน OOP ที่ประกาศวิธีการที่เป็นนามธรรมอย่างน้อยหนึ่งวิธี คลาสเหล่านี้สามารถมีวิธีการที่เป็นนามธรรมและวิธีการที่เป็นรูปธรรม คลาสปกติไม่สามารถมีเมธอดนามธรรมได้ คลาสนามธรรมคือคลาสที่มีเมธอดนามธรรมอย่างน้อยหนึ่งวิธี



Abstract Method คืออะไร?

- Abstract Method เป็นเมธอดที่มีแต่นิยาม แต่ไม่มีการระบุถึงวิธีการทำงาน (implementation)
- เมธอดที่ไม่มีส่วนที่เป็น body เป็นที่รู้จักคือ Abstract Method
- Abstract Method จะต้องประกาศใน abstract class
- Abstract Method จะไม่ถูกประกาศด้วยคีย์เวิร์ด final เนื่องจาก abstract class ต้องใช้งาน Abstract Method ทั้งหมด



ข้อดีของ *Abstraction*

- ประโยชน์หลักของการใช้ *Abstraction* ในการเขียนโปรแกรมคือช่วยให้คุณสามารถจัดกลุ่มคลาสที่เกี่ยวข้องหลายคลาสเป็นพี่น้องได้
- *Abstraction* ในการเขียนโปรแกรม OOP ช่วยลดความซับซ้อนของกระบวนการออกแบบและการใช้งานซอฟต์แวร์



เมื่อใดควรใช้ *Abstract Methods* และ *Abstract Class*

Abstract Methods ส่วนใหญ่จะประกาศโดยที่คลาสย่อยตั้งแต่สองคลาสขึ้นไปทำสิ่งเดียวกันในรูปแบบที่แตกต่างกันผ่านการนำไปใช้งานที่แตกต่างกัน

นอกจากนี้ Abstract Class เดียวกัน สามารถมีวิธีการทำงานของ abstract method ที่แตกต่างกันได้

Abstract Class จะช่วยอธิบายลักษณะพฤติกรรมการทำงานทั่วไปและลำดับชั้นของ object-oriented programming นอกจากนี้ Abstract Class ยังจะช่วยอธิบายรายละเอียดการทำงานของ subclass ต่าง ๆ ของ Abstract Class



Abstract คีย์เวิร์ด

Abstract data เป็นกระบวนการในการซ่อนรายละเอียดบางอย่างและแสดงเฉพาะข้อมูลที่จำเป็นต่อผู้ใช้ **Abstract** สามารถทำได้ด้วย **Abstract class** หรืออินเทอร์เฟซ

คีย์เวิร์ด **abstract** คือ non-access modifier ที่ใช้สำหรับคลาสและเมธอด:

Abstract class : เป็นคลาสที่ถูกจำกัด ซึ่งไม่สามารถใช้สร้างอ็อบเจกต์ได้ (หากต้องการเข้าถึง คลาสนั้นต้องสืบทอดมาจากคลาสอื่น)

Abstract method : ใช้ได้เฉพาะใน **Abstract class** และไม่มีเนื้อหาหรือส่วนที่เป็น body ของโค้ด ส่วน body ของโค้ดมีให้โดย subclass (class ที่สืบทอดมาจาก **Abstract class**)

Abstract class สามารถมีได้ทั้ง เมธอดที่เป็น **Abstract** และ เมธอดปกติ



ตัวอย่าง *Abstract class* มีทั้ง เมธอดที่เป็น *Abstract* และ เมธอดปกติ

```
abstract class Animal
{
    public abstract void animalSound();

    public void sleep()
    {
        System.out.println("Zzz");
    }
}
```

จากตัวอย่างข้างต้น ไม่สามารถสร้างอ็อบเจกต์
ของคลาส Animal ได้:

```
Animal myObj = new Animal(); // will generate
                                //an error
```

ในการเข้าถึง *Abstract class* จะต้องสืบทอดมา
จากคลาสอื่น มาแปลงคลาส Animal เป็น
Abstract class กัน



ตัวอย่าง abstract class

```
// Main.java
// Abstract class
abstract class Animal
{
    // Abstract method (does not have a body)
    public abstract void animalSound();
    // Regular method
    public void sleep() {
        System.out.println("Zzz");
    }
}

// Subclass (inherit from Animal)
class Pig extends Animal
{
    public void animalSound()
    {
        // The body of animalSound() is provided here
        System.out.println("The pig says: wee wee");
    }
}

class Main
{
    public static void main(String[] args)
    {
        Pig myPig = new Pig(); // Create a Pig object
        myPig.animalSound();
        myPig.sleep();
    }
}
```



โค้ด Main.java

<https://onlinegdb.com/KPOobeY52>

```
The pig says: wee wee
Zzz
```



ทำไมและเมื่อใดจึงควรใช้ *Abstract class* และ *Abstract Methods*

เพื่อให้ได้ความปลอดภัย – ซ่อนรายละเอียดบางอย่างและแสดงเฉพาะรายละเอียดที่สำคัญของอ็อบเจกต์

หมายเหตุ: คุณสามารถเข้าถึงสิ่งที่เป็น *Abstract* ได้ด้วยอินเทอร์เฟซ ซึ่งคุณสามารถเรียนรู้เพิ่มเติมเนื้อหาถัดไป



อะไรคือ Encapsulation ใน Java?

Encapsulation ใน Java คืออะไร Encapsulation ใน Java เป็นกลไกในการรวมตัวแปร (ข้อมูล) และวิธีการ (เมธอด) เข้าด้วยกันเป็นหน่วยเดียว เป็นกระบวนการซ่อนรายละเอียดข้อมูลและปกป้องข้อมูลและพฤติกรรมของอ็อบเจกต์ เป็นหนึ่งในสี่แนวคิด OOP ที่สำคัญ คลาสการห่อหุ้มนั้นง่ายต่อการทดสอบ ดังนั้นจึงเป็นการดีกว่าสำหรับการทดสอบหน่วยด้วย



ตัวอย่าง

เพื่อให้เข้าใจว่าการห่อหุ้มโดยละเอียดคืออะไร ให้พิจารณาประเภทบัญชีธนาคาร
ต่อไปนี้พร้อมวิธีการฝากและแสดงยอดคงเหลือ

```
class Account {  
    private int account_number;  
    private int account_balance;  
  
    public void show Data() {  
        //code to show data  
    }  
  
    public void deposit(int a) {  
        if (a < 0) {  
            //show error  
        } else  
            account_balance = account_balance + a;  
    }  
}
```



ตัวอย่าง (ต่อ)

สมมติว่าแฮ็กเกอร์สามารถเข้าถึงรหัสบัญชีธนาคารของคุณได้ ตอนนี้เขาพยายามฝากเงินจำนวน -100 เข้าบัญชีของคุณด้วยสองวิธี ให้ดูวิธีการหรือแนวทางแรกของเขา

```
class Hacker{  
    Account a= new Account ();  
    a.account_balance= -100;  
}
```

แนวทางที่ 1: เขาพยายามฝากจำนวนเงินที่ไม่ถูกต้อง (เช่น -100) เข้าบัญชีธนาคารของคุณ โดยจัดการรหัส

ตอนนี้คำถามคือ - เป็นไปได้ไหม? ให้สอบสวนดู โดยปกติ ตัวแปรในคลาสจะถูกตั้งค่าเป็น “private” ดังที่แสดงด้านล่าง สามารถเข้าถึงได้ด้วยวิธีการที่กำหนดไว้ในคลาสนั้นเท่านั้น ไม่มีคลาสหรืออ็อบเจกต์อื่นใดที่สามารถเข้าถึงได้

```
class Account{  
    private int account_number;  
    private int account_balance;  
  
    public void show Data(){  
        // code to show data  
    }  
  
    public void deposit(int a){
```

หากสมาชิกข้อมูลเป็นแบบ private แสดงว่าสามารถเข้าถึงได้ภายในคลาสดียวกันเท่านั้น ไม่มีคลาสภายนอกที่สามารถเข้าถึงสมาชิกข้อมูลที่เป็น private หรือตัวแปรของคลาสอื่นได้ ดังนั้นในกรณีของเรา แฮ็กเกอร์ไม่สามารถฝากเงินจำนวน -100 เข้าบัญชีของคุณได้

```
class Hacker{  
    Account a= new Account ();  
    a.account_balance= -100;  
}
```



ตัวอย่าง (ต่อ)

แนวทางที่ 2: วิธีแรกของแฮกเกอร์ไม่สามารถฝากเงินได้ ต่อไปเขาพยายามฝากเงินจำนวน -100 โดยใช้เมธอด “deposit-ฝาก”

```
class Hacker{  
    Account a= new Account  
    a.account_balance= -100  
    a.deposit(-100);  
}
```

แต่การใช้เมธอดมีการตรวจสอบค่าลบ ดังนั้นวิธีที่สองก็ล้มเหลวเช่นกัน

```
public void deposit(int a){  
    if (a<0){  
        //show error  
    }  
    else  
        account_balance=  
    }  
}
```

method
implementation has
checked for
negative values (-100
) and throws an

ดังนั้น คุณไม่เคยเปิดเผยข้อมูลของคุณกับบุคคลภายนอก
ซึ่งทำให้แอปพลิเคชันของคุณปลอดภัย

```
class Account{  
    private int account_number;  
    private int account_balance;  
  
    public void show Data(){  
        // code to show data  
    }  
  
    public void deposit(int a){  
        if (a<0){  
            //show error  
        }  
        else  
            account_balance= account_balance+ a,  
        }  
    }
```

The entire code can
be thought of a
capsule, and can
be only
communicate
through messages

รหัสทั้งหมดสามารถคิดได้ว่าเป็นแคปซูล และคุณสามารถสื่อสารผ่านข้อความเท่านั้น จึงเป็นที่มาของ
ชื่อ การห่อหุ้ม(encapsulation)



การซ่อนข้อมูลใน Java

Getter และ Setter ใน Java เป็นวิธีการทั่วไปสองวิธีที่ใช้ในการดึงและอัปเดตค่าของตัวแปร ส่วนใหญ่จะใช้ในการสร้าง แก้ไข ลบ และดูค่าตัวแปร เมธอด setter ใช้สำหรับอัปเดตค่า และเมธอด getter ใช้สำหรับอ่านหรือดึงค่า พวกเขายังเป็นที่รู้จักกันในนาม accessor และ mutator



ตัวอย่างการซ่อนข้อมูล

ในตัวอย่างข้างต้น เมธอด `getBalance()` คือเมธอด `getter` ที่อ่านค่าของตัวแปร `account_balance` และเมธอด `setNumber()` คือเมธอด `setter` ที่ตั้งค่าหรืออัปเดตค่าสำหรับตัวแปร `account_number`

```
class Account{  
    private int account_number;  
    private int account_balance;  
    // getter method  
    public int getBalance() {  
        return this.account_balance;  
    }  
    // setter method  
    public void setNumber(int num) {  
        this.account_number = num;  
    }  
}
```



Abstraction กับ Encapsulation

การห่อหุ้ม(**Encapsulation**) มักเข้าใจผิดกับสิ่งที่เป็่น **Abstraction**

Encapsulation เป็นเรื่องเกี่ยวกับ “How”-“อย่างไร” เพื่อให้ฟังก์ชันบรรลุการทำงาน

Abstraction เป็นมากกว่า “What”-“อะไร” ที่คลาสสามารถทำได้

ตัวอย่างง่ายๆ ในการทำความเข้าใจความแตกต่างนี้คือโทรศัพท์มือถือ โดยที่ตรรกะที่
ซับซ้อนในแผงวงจรถูกห่อหุ้ม(**Encapsulation**) ไว้ในหน้าจอสัมผัส และอินเทอร์เฟซมี
ไว้เพื่อให้เป็น **Abstraction**



ข้อดีของ Encapsulation ในภาษา Java

- การห่อหุ้มจะผูกข้อมูลกับฟังก์ชันที่เกี่ยวข้อง ฟังก์ชันนี้หมายถึง “เมธอด” และข้อมูลหมายถึง “ตัวแปร”
- ดังนั้นเราจึงเก็บตัวแปรและเมธอดไว้ในที่เดียวกัน ในที่นี้คือ “คลาส” คลาสเป็นฐานสำหรับการห่อหุ้ม
- ด้วย Java Encapsulation คุณสามารถซ่อน (จำกัดการเข้าถึง) เฉพาะสมาชิกข้อมูลที่สำคัญในโค้ดของคุณ ซึ่งช่วยเพิ่มความปลอดภัย
- ดังที่เราได้กล่าวไว้ก่อนหน้านี้ หากสมาชิกข้อมูลได้รับการประกาศว่าเป็น “private-ส่วนตัว” จะสามารถเข้าถึงได้ภายในคลาสเดียวกันเท่านั้น ไม่มีคลาสภายนอกที่สามารถเข้าถึงข้อมูลสมาชิก (ตัวแปร) ของคลาสอื่น
- อย่างไรก็ตาม หากคุณต้องการเข้าถึงตัวแปรเหล่านี้ คุณต้องใช้วิธี “getter” และ “setter” แบบสาธารณะ-public

The left side of the slide features several overlapping, semi-transparent geometric shapes, primarily triangles and quadrilaterals, in shades of gray. These shapes are arranged in a way that creates a modern, abstract background element.

ตัวแปรใน Java

(Java Variables)



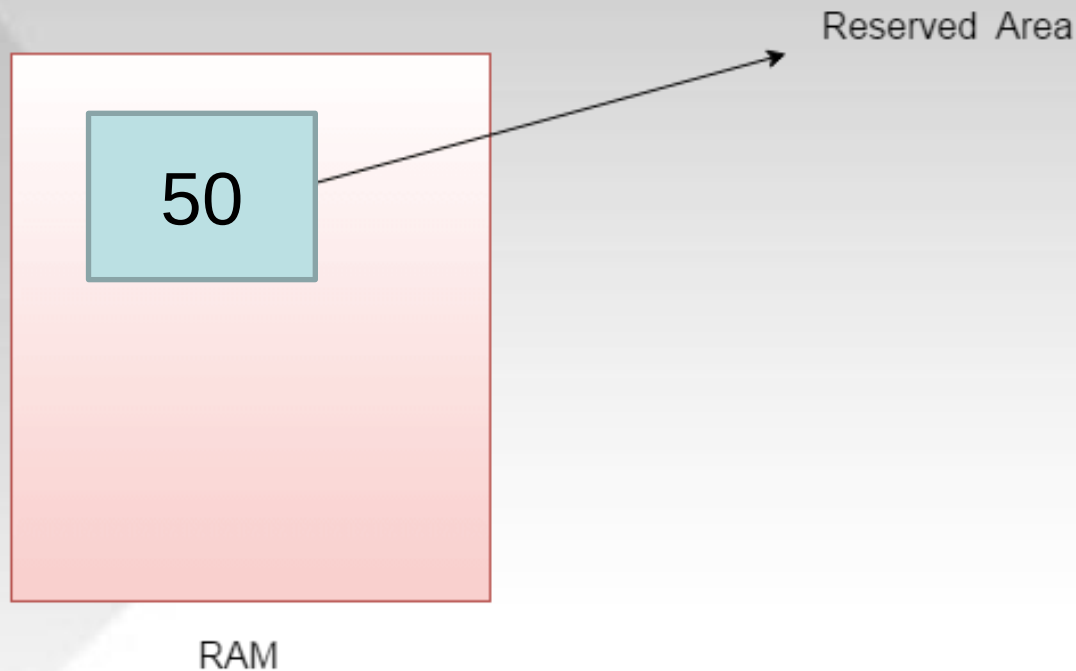
Java Variables

- ตัวแปรคือคอนเทนเนอร์ที่เก็บค่าในขณะที่รันโปรแกรม Java
- ตัวแปรถูกกำหนดด้วยชนิดข้อมูล Variable คือชื่อของตำแหน่งหน่วยความจำ
- ตัวแปรใน java มีสามประเภท: โลคัล-local , อินสแตนซ์- instance , และสแตติก-static
- ประเภทของข้อมูลใน Java มี 2 ประเภท: primitive และ non-primitive



Variable

ตัวแปรคือชื่อของพื้นที่สงวนที่จัดสรรในหน่วยความจำ กล่าวอีกนัยหนึ่งก็คือชื่อของตำแหน่งหน่วยความจำ เป็นการผสมผสานระหว่าง "vary + can" ซึ่งหมายความว่าค่าของมันสามารถเปลี่ยนแปลงได้ เช่น `int data=50; //data is variable`





ตัวแปร Java และประเภทข้อมูล

ตัวแปรใน Java คือที่เก็บข้อมูลที่เก็บค่าข้อมูลระหว่างการทำงานของโปรแกรม Java
ตัวแปรทุกตัวถูกกำหนดประเภทข้อมูลซึ่งกำหนดประเภทและปริมาณของค่าที่สามารถ
เก็บได้

Variable คือชื่อตำแหน่งหน่วยความจำของข้อมูล ตัวแปร Java ส่วนใหญ่มีสามประเภท:
Local, Instance และ Static

ในการใช้ตัวแปรในโปรแกรม คุณต้องทำ 2 ขั้นตอน

- การประกาศตัวแปร
- การกำหนดค่าเริ่มต้นให้ตัวแปร



การประกาศตัวแปร

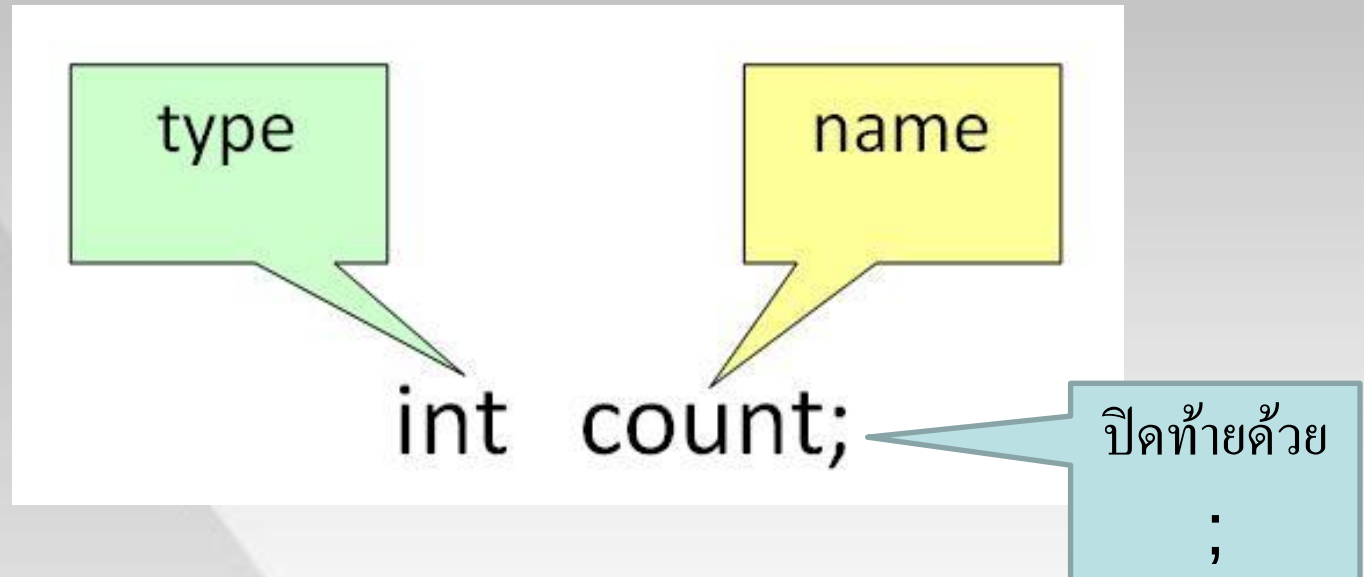
หากต้องการประกาศตัวแปร คุณต้องระบุประเภทข้อมูลและตั้งชื่อเฉพาะให้ตัวแปร

```
int a,b,c;
```

```
float pi;
```

```
double d;
```

```
char a;
```





การกำหนดค่าเริ่มต้นให้ตัวแปร:

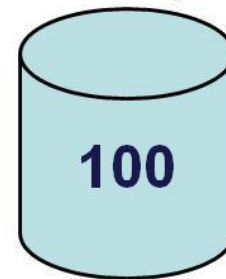
ในการกำหนดค่าเริ่มต้นให้ตัวแปร คุณต้องกำหนดค่าที่ถูกต้องให้กับตัวแปร

```
pi = 3.14f;
```

```
do = 20.22d;
```

```
a = 'v';
```

```
count = 100;
```



Container named
"Count" holding
a value 100



การประกาศตัวแปรและค่าเริ่มต้นพร้อมกัน

คุณสามารถรวมการประกาศตัวแปรและกำหนดค่าเริ่มต้นให้ตัวแปรได้พร้อมกัน

```
int a=2,b=4,c=6;
```

```
float pi=3.14f;
```

```
double do=20.22d;
```

```
char a='v';
```

```
int count = 100;
```

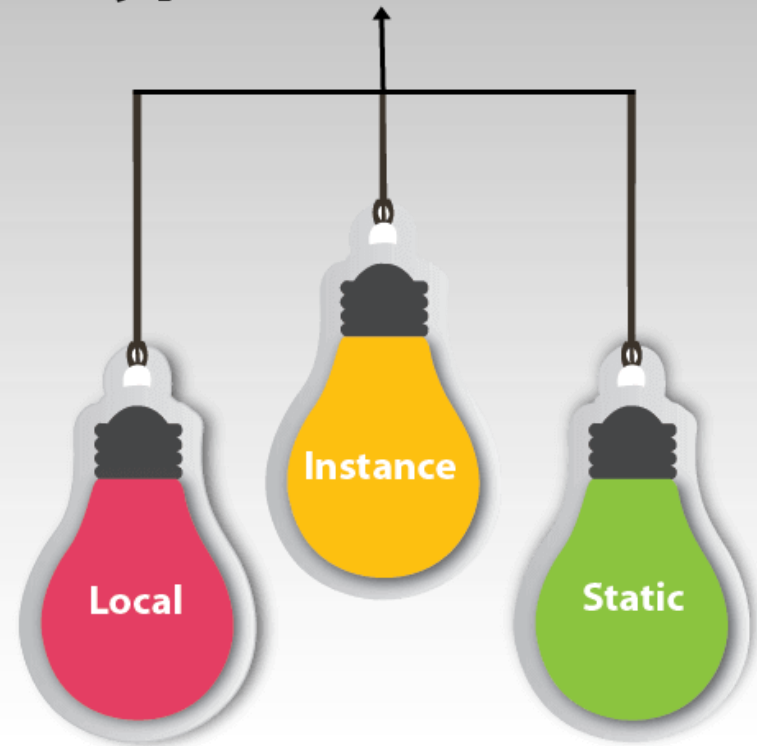



ประเภทของตัวแปร-Types of Variables

Java มีตัวแปรสามประเภท :

- ตัวแปรท้องถิ่น- local variable
- ตัวแปรอินสแตนซ์- instance variable
- ตัวแปรคงที่- static variable

Types of Variables





ประเภทของตัวแปร (ต่อ)

1) ตัวแปรท้องถิ่น

Local Variables เป็นตัวแปรที่ถูกประกาศไว้ภายในเมธอด

2) ตัวแปรอินสแตนท์

ตัวแปรอินสแตนท์ถูกกำหนดโดยไม่มีคีย์เวิร์ด STATIC พวกมันถูกกำหนดไว้นอกเมธอด เป็นอ็อบเจกต์เฉพาะและเรียกว่าตัวแปรอินสแตนท์

3) ตัวแปรคงที่

ตัวแปรสแตติกจะเริ่มขึ้นเพียงครั้งเดียว เมื่อเริ่มต้นการทำงานของโปรแกรม ตัวแปรเหล่านี้ควรเริ่มต้นก่อน ก่อนการเริ่มต้นของตัวแปรอินสแตนท์ใดๆ



ตัวอย่าง

```
class Guru99 {  
    static int a = 1; //static variable  
    int data = 99; //instance variable  
    void method() {  
        int b = 90; //local variable  
    }  
}
```



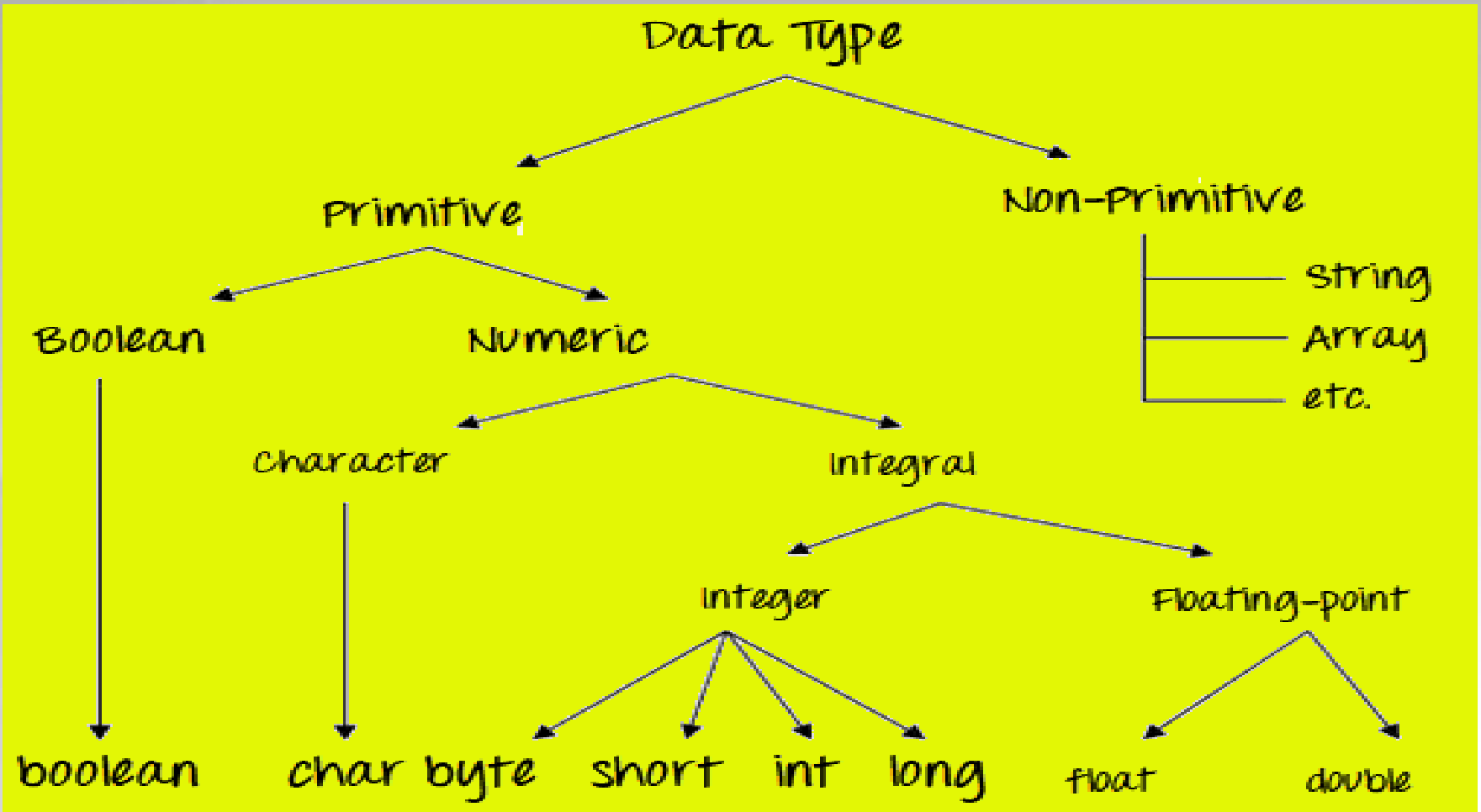
ประเภทข้อมูลใน Java คืออะไร?

ประเภทข้อมูลใน Java ถูกกำหนดเป็นตัวระบุที่จัดสรรขนาดและประเภทของค่าต่างๆ ที่สามารถเก็บไว้ในตัวแปรหรือตัวระบุได้ Java มีชุดข้อมูลหลายประเภท ชนิดข้อมูลใน Java สามารถแบ่งออกเป็นสองส่วน:

- Primitive Data Types :- ซึ่งรวมถึงจำนวนเต็ม อักขระ บูลีน และ float
- Non-primitive Data Types :- ซึ่งรวมถึงคลาส อาร์เรย์ และอินเทอร์เฟซ



ประเภทข้อมูลใน Java คืออะไร? (ต่อ)





Primitive Data Types-ชนิดข้อมูลดั้งเดิม

Primitive Data Types ถูกกำหนดไว้ล่วงหน้าและพร้อมใช้งานในภาษา Java ค่าดั้งเดิม
ไม่แบ่งปันสถานะกับค่าดั้งเดิมอื่นๆ

มี 8 ประเภทพื้นฐาน: byte , short , int , long , char, float, double และ Boolean

Integer data types

byte (1 byte)
short (2 bytes)
int (4 bytes)
long (8 bytes)

Floating Data Type

float (4 bytes)
double (8 bytes)

Textual Data Type

char (2 bytes)

Logical

boolean (1 byte) (true/false)



Java Data Types

Data Type	Default Value	Default size
byte	0	1 byte
short	0	2 bytes
int	0	4 bytes
long	0L	8 bytes
float	0.0f	4 bytes
double	0.0d	8 bytes
boolean	false	1 bit
char	'\u0000'	2 bytes



จุดที่ต้องจำ

- ประเภทข้อมูลตัวเลขทั้งหมดเป็นแบบคิดเครื่องหมาย (+/-)
- ขนาดของประเภทข้อมูลยังคงเท่าเดิมเสมอในทุกแพลตฟอร์ม (เป็นค่ามาตรฐาน)
- ชนิดข้อมูล char ใน Java มีขนาด 2 ไบต์เพราะใช้ชุดอักขระ UNICODE ซึ่งทำให้ Java รองรับการทำให้เป็นสากล(internationalization) UNICODE เป็นชุดอักขระที่ครอบคลุมสคริปต์และภาษาที่รู้จักทั้งหมดในโลก



จุดที่ต้องจำ

- ประเภทข้อมูลตัวเลขทั้งหมดเป็นแบบคิดเครื่องหมาย (+/-)
- ขนาดของประเภทข้อมูลยังคงเท่าเดิมเสมอในทุกแพลตฟอร์ม (เป็นค่ามาตรฐาน)
- ชนิดข้อมูล char ใน Java มีขนาด 2 ไบต์เพราะใช้ชุดอักขระ UNICODE ซึ่งทำให้ Java รองรับการทำให้เป็นสากล(internationalization) UNICODE เป็นชุดอักขระที่ครอบคลุมสคริปต์และภาษาที่รู้จักทั้งหมดในโลก



Type Conversion และ Type Casting ใน Java

Type Casting เป็นการกำหนดค่าของ Variable ที่มี Data **Type** ที่เราไม่ต้องการ ให้กับ Variable ที่มี Data **Type** ที่เราต้องการ การทำ **Type Casting** นั้น Data **Type** ทั้งสองจำเป็น สอดคล้องกัน เช่น เป็น Data **Type** ที่เก็บตัวเลขเหมือนกัน

Type Conversion(Casting) คือ การที่ผู้เขียนโปรแกรมทำการเปลี่ยนแปลงชนิดข้อมูลจากชนิดหนึ่งไป เป็นอีกชนิดหนึ่งด้วยตัวเอง



การแปลงประเภทตัวแปรและการหล่อประเภท-Type Conversion & Type Casting ใน Java

ตัวแปรประเภทหนึ่งสามารถรับค่าของอีกประเภทหนึ่งได้ มี 2 กรณีคือ

กรณีที่ 1) ตัวแปรของความจุที่น้อยกว่าถูกกำหนดให้กับตัวแปรอื่นที่มีความจุมากกว่า

กระบวนการนี้เป็นแบบอัตโนมัติ และเรียกว่า Conversion

```
double d ;  
int i = 10;  
d = i;
```

กรณีที่ 2) ตัวแปรของความจุที่มากขึ้นถูกกำหนดให้กับตัวแปรอื่นที่มี

ความจุน้อยกว่า ในกรณีดังกล่าว คุณต้องระบุ type cast operator ให้ชัดเจน

กระบวนการนี้เรียกว่า Type Casting

```
double d = 10;  
int i;  
i = (int) d
```

Type Cast
Operator

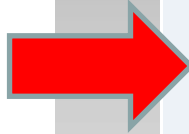
ในกรณีที่คุณไม่ได้ระบุตัวดำเนินการประเภทการแคสต์ คอมไพเลอร์จะให้ข้อผิดพลาด เนื่องจากกฎนี้บังคับใช้โดยคอมไพเลอร์ มันทำให้โปรแกรมเมอร์ทราบว่าการแปลงที่เขา กำลังจะทำอาจทำให้ข้อมูลสูญหายและป้องกันการสูญเสียโดยไม่ได้ตั้งใจ



ตัวอย่าง

```
1 class DemoType
2 {
3     public static void main(String args[])
4     {
5         byte x;
6         int a = 270;
7         double b = 128.128;
8
9         System.out.println("int converted to byte");
10        x = (byte) a;
11        System.out.println("a and x " + a + " " + x);
12        System.out.println("double converted to int");
13        a = (int) b;
14        System.out.println("b and a " + b + " " + a);
15        System.out.println("\ndouble converted to byte");
16        x = (byte)b;
17        System.out.println("b and x " + b + " " + x);
18    }
19 }
```

Output:



```
int converted to byte
a and x 270 14
double converted to int
b and a 128.128 128

double converted to byte
b and x 128.128 -128
```

คลาสและออบเจกต์ใน Java

(Class and Object in Java)



What is Class and Object in Java OOPS

คลาสและอ็อบเจกต์ใน Java เป็นองค์ประกอบพื้นฐานของ OOP มักจะมีความสัมพันธ์ระหว่างคลาส และอ็อบเจกต์ เราพยายามบอกคุณถึงความแตกต่างระหว่าง คลาสและอ็อบเจกต์ ใน Java



คลาสใน Java คืออะไร?

คลาสเป็นพิมพ์เขียวหรือชุดคำสั่งในการสร้างอ็อบเจกต์ประเภทใดประเภทหนึ่ง เป็นแนวคิดพื้นฐานของ Object-Oriented Programming ซึ่งหมุนรอบเอนทิตีในชีวิตจริง คลาสใน Java กำหนดวิธีการทำงานของอ็อบเจกต์และสิ่งที่อ็อบเจกต์จะมี

Syntax

```
class <class_name>{  
    field;  
    method;  
}
```



Object ใน Java คืออะไร?

อ็อบเจกต์ เป็นแทนของคลาส อ็อบเจกต์ใน OOPS ไม่ได้เป็นเพียงแค่องค์ประกอบที่มีอยู่ในตัว ซึ่งประกอบด้วยวิธีการและคุณลักษณะในการทำให้ข้อมูลบางประเภทมีประโยชน์ เช่น ชื่อสี โต๊ะ กระเป๋า เหา เมื่อคุณส่งข้อความไปยังอ็อบเจกต์ คุณกำลังขอให้อ็อบเจกต์เรียกใช้หรือดำเนินการหนึ่งในเมธอดตามที่กำหนดไว้ในคลาส

จากมุมมองของการเขียนโปรแกรม อ็อบเจกต์ใน OOPS สามารถรวมโครงสร้างข้อมูลตัวแปร หรือฟังก์ชันได้ มีการจัดสรรตำแหน่งหน่วยความจำ Java Objects ได้รับการออกแบบเป็นลำดับชั้นของคลาส (class hierarchies)

Syntax

```
ClassName ReferenceVariable = new ClassName();
```




ความแตกต่างระหว่างอ็อบเจกต์และคลาสใน Java

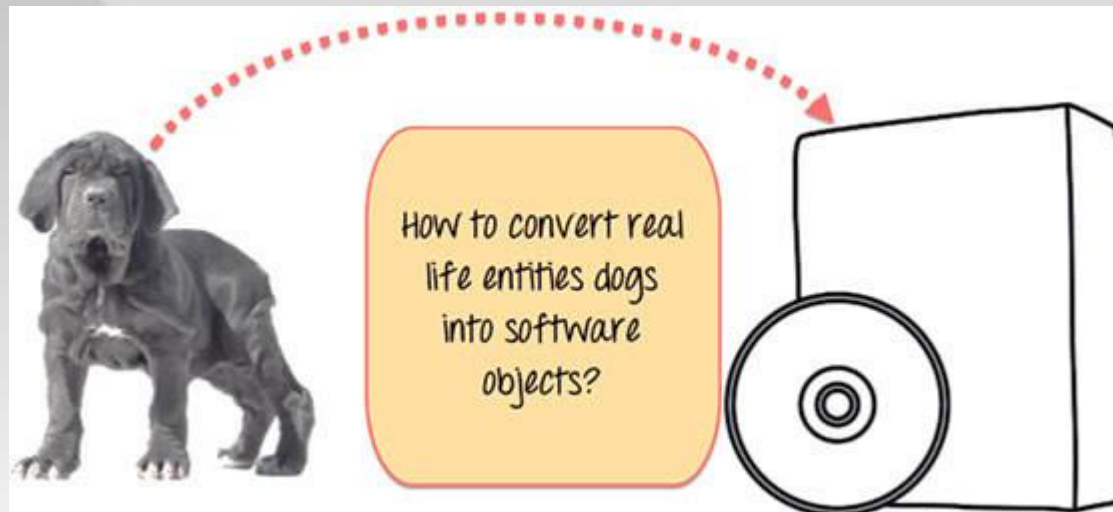
คลาสในการเขียนโปรแกรมเชิงอ็อบเจกต์-OOP เป็นพิมพ์เขียวหรือต้นแบบที่กำหนดตัวแปรและวิธีการ (ฟังก์ชัน) ทั่วไปของ Java Objects บางชนิด

อ็อบเจกต์ใน OOPS เป็นตัวอย่างของคลาส อ็อบเจกต์ซอฟต์แวร์มักใช้เพื่อจำลองอ็อบเจกต์ในโลกแห่งความเป็นจริงที่คุณพบในชีวิตประจำวัน



ทำความเข้าใจแนวคิดของ Java Classes และ Objects ด้วยตัวอย่าง

มาดูตัวอย่างการพัฒนาระบบการจัดการสัตว์เลี้ยงสำหรับสุนัขโดยเฉพาะ คุณจะต้องใช้ข้อมูลต่างๆ เกี่ยวกับสุนัข เช่น สุนัขสายพันธุ์ต่างๆ อายุ ขนาด ฯลฯ คุณต้องจำลองสิ่งมีชีวิตในชีวิตจริง เช่น สุนัขให้กลายเป็นซอฟต์แวร์ คำถามคือ คุณจะออกแบบซอฟต์แวร์ดังกล่าวอย่างไร





ทำความเข้าใจแนวคิดของ Java Classes และ Objects ด้วยตัวอย่าง (ต่อ)

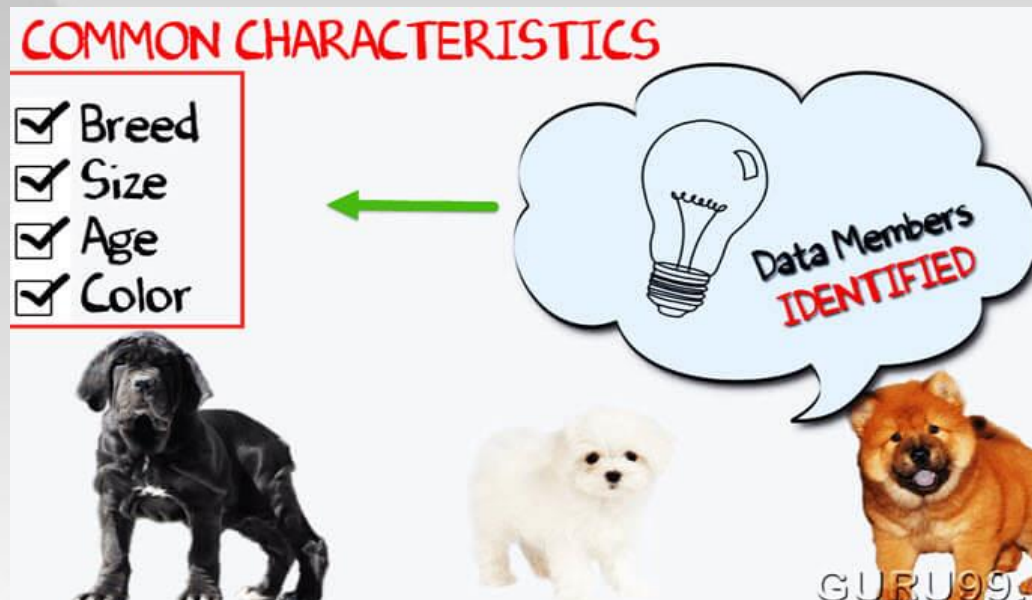
ก่อนอื่นมาตอบคำถามกันก่อน ให้คุณดูภาพสุนัขสามสายพันธุ์ที่แตกต่างกันด้านล่าง





ทำความเข้าใจแนวคิดของ Java Classes และ Objects ด้วยตัวอย่าง (ต่อ)

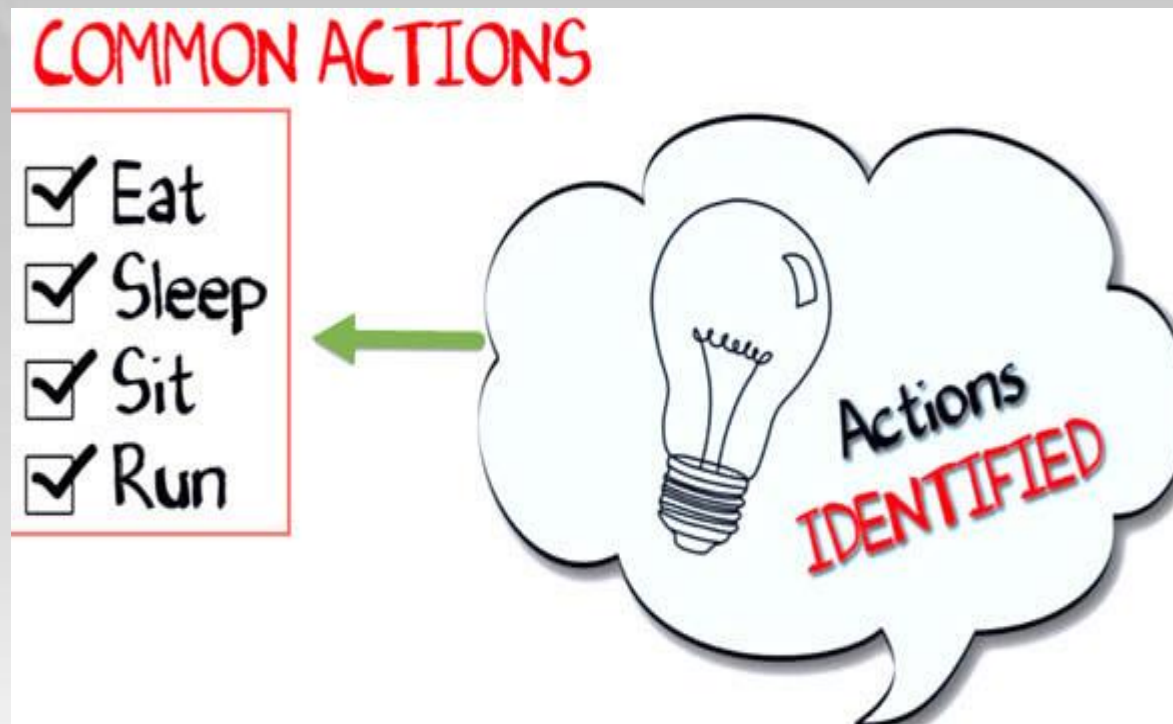
หยุดตรงนี้เดี๋ยวก่อน! ลงรายการความแตกต่างระหว่างสุนัข ความแตกต่างบางอย่างที่คุณอาจระบุไว้ อาจเป็นสายพันธุ์ อายุ ขนาด สี ฯลฯ หากคุณคิดสักครู่ ความแตกต่างเหล่านี้เป็นลักษณะทั่วไปบางประการที่สุนัขเหล่านี้มีร่วมกัน ลักษณะเหล่านี้ (สายพันธุ์ อายุ ขนาด สี) สามารถสร้างสมาชิกของข้อมูลสำหรับอ็อบเจกต์ของคุณ





ทำความเข้าใจแนวคิดของ Java Classes และ Objects ด้วยตัวอย่าง (ต่อ)

ต่อไป ระบุพฤติกรรมทั่วไปของสุนัขเหล่านี้ เช่น นอน นั่ง กิน ฯลฯ ดังนั้นสิ่งเหล่านี้จะ
เป็นการกระทำของออบเจกต์ซอฟต์แวร์ของเรา





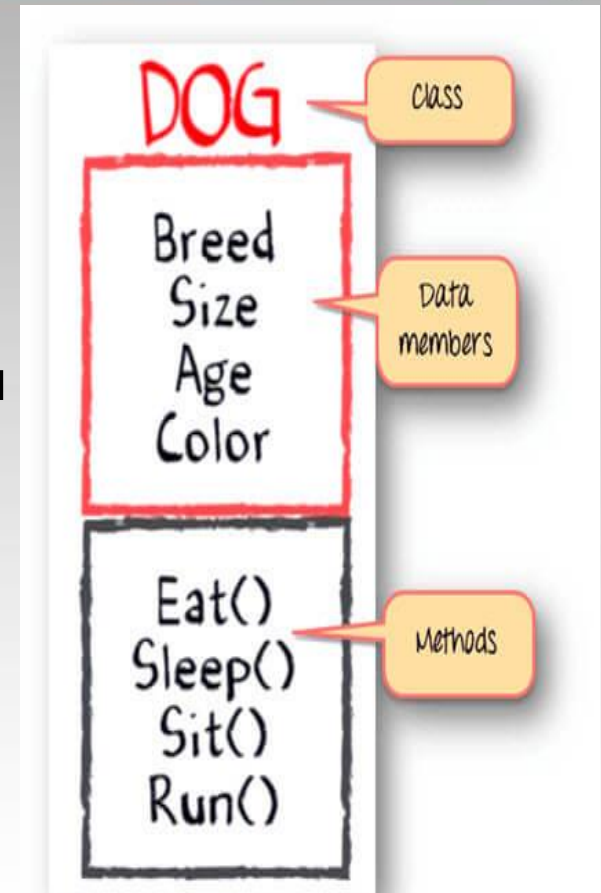
ทำความเข้าใจแนวคิดของ Java Classes และ Objects ด้วยตัวอย่าง (ต่อ)

จนถึงตอนนี้เราได้กำหนดสิ่งต่อไปนี้

Class-คลาส – Dogs

Data members-สมาชิกข้อมูล - ขนาด อายุ สี สายพันธุ์ ฯลฯ

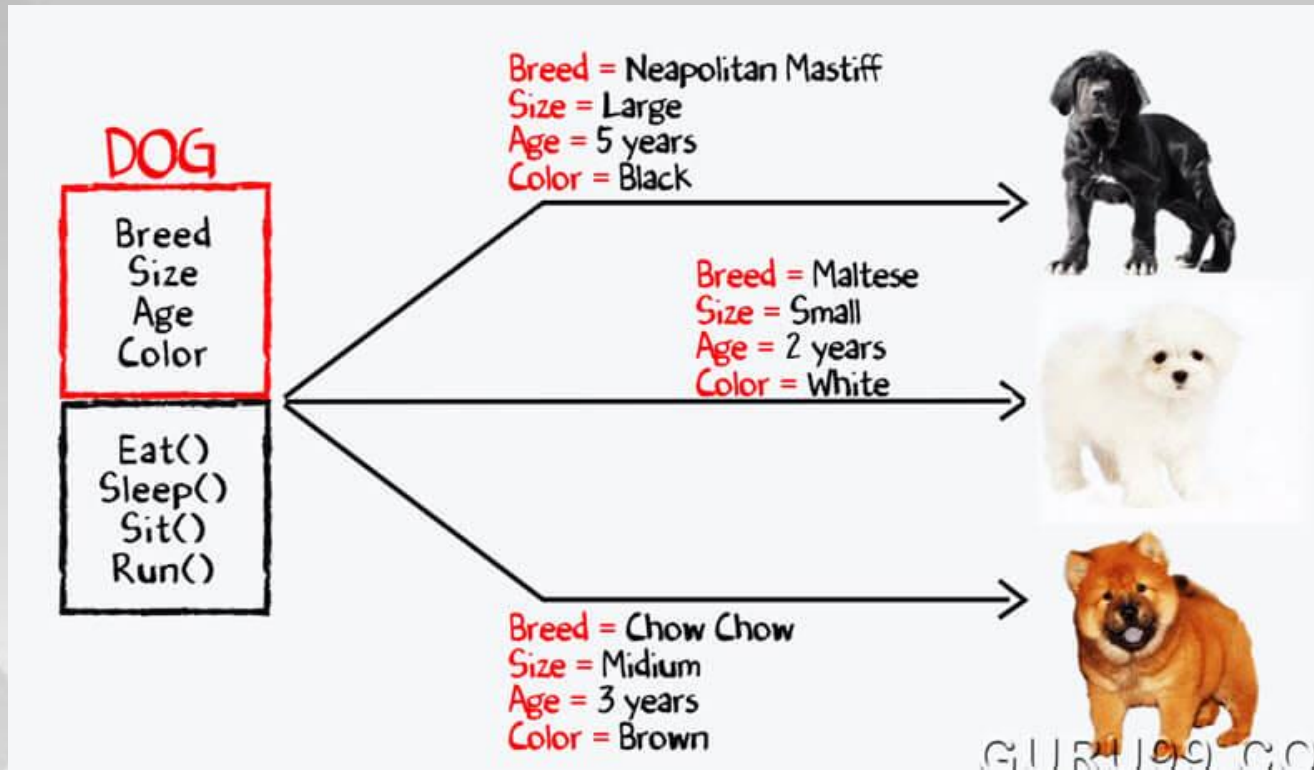
Method-พฤติกรรมต่าง ๆ – กิน นอน นั่ง และวิ่ง





ทำความเข้าใจแนวคิดของ Java Classes และ Objects ด้วยตัวอย่าง (ต่อ)

ตอนนี้ สำหรับค่าต่างๆ ของสมาชิกข้อมูล (ขนาดพันธุ์ อายุ และสี) ในคลาส Java คุณจะ
ได้รับอ็อบเจกต์ dog ที่แตกต่างกัน





ทำความเข้าใจแนวคิดของ Java Classes และ Objects ด้วยตัวอย่าง (ต่อ)

คุณสามารถออกแบบโปรแกรมใดก็ได้โดยใช้แนวทาง OOPs นี้

ขณะสร้างคลาส เราต้องปฏิบัติตามหลักการดังต่อไปนี้

- Single Responsibility Principle (SRP)- คลาสควรมีเหตุผลเพียงข้อเดียวในการเปลี่ยนแปลง
- Open Closed Responsibility (OCP) - ควรขยายคลาสใด ๆ โดยไม่ต้องแก้ไข
- Liskov Substitution Responsibility (LSR)- คลาสที่รับจะต้องใช้แทนคลาสพื้นฐานได้
- Dependency Inversion Principle(DIP)- ขึ้นอยู่กับสิ่งที่เป็นามธรรมและไม่ใช่งการยัดคิด
- Interface Segregation Principle (ISP) - เตรียมอินเทอร์เฟซแบบละเอียดที่เจาะจงเฉพาะไคลเอนต์



โปรแกรมตัวอย่าง คลาสและอ็อบเจกต์ใน Java

```
1 //Dog.java
2 // Class Declaration
3 public class Dog {
4     // Instance Variables
5     String breed;
6     String size;
7     int age;
8     String color;
9
10
11     // method 1
12     public String getInfo() {
13         return ("Breed is: "+breed+" Size is:"+size+" Age is:"+age+" color is: "+color);
14     }
15
16
17     public static void main(String[] args) {
18         Dog maltese = new Dog();
19         maltese.breed="Maltese";
20         maltese.size="Small";
21         maltese.age=2;
22         maltese.color="white";
23         System.out.println(maltese.getInfo());
24     }
25 }
```

Output:

```
Breed is: Maltese Size is:Small Age is:2 color is: white
```



Java Object และ Class Example: main ภายนอก class

```
1 //Dog2.java
2 // Class Declaration
3 class Dog2 {
4     // Instance Variables
5     String breed;
6     String size;
7     int age;
8     String color;
9
10
11     // method 1
12     public String getInfo() {
13         return ("Breed is: "+breed+" Size is:"+size+" Age is:"+age+" color is: "+color);
14     }
15 }
16 public class Execute{
17     public static void main(String[] args) {
18         Dog maltese = new Dog();
19         maltese.breed="Maltese";
20         maltese.size="Small";
21         maltese.age=2;
22         maltese.color="white";
23         System.out.println(maltese.getInfo());
24     }
25 }
```

Output:

```
Breed is: Maltese Size is:Small Age is:2 color is: white
```



Q & A

